08장

라이브러리 — 기능. 만들어져 있는 부품 — 클래스, 함수, 변수

랭체인으로 에이전트 만들기

App + LLM

이번 장에서는 랭체인으로 직접 에이전트를 개발하며, 오픈AI의 GPT API만 사용해 개발할 때와 어떤 공통점과 차이점이 있는지 경험해보겠습니다.

LC이용해서 에이전트 개발 5단계

GPT API -----> 코드

claude API ----> 코드

변경 어렵. LangChain은 가능

8장의 목적
- 현재까지는 GPT API로 직접 코딩(대화 기억, 함수 호출, 스트림)
  -> LangChain 라이브러리(부품)로 코딩하는 방법 배울 것.
  tool을 스스로 골라 작동하는 chatbot 개발(웹)

Agent 개발 5단계(흐름)
1. 대화
   chatOpenAI(LangChain)
   (메시지, 기억 담당)
2. 체인(연결)
   프롬프트, 모델, 파서
3. 도구
   - @tool -> 함수에 붙임.
   - bind_tools
   - 도구 순환
4. Stream
   - 조각(chunk) 모아붙이기(답변 text)
5. 화면
   - 웹 사용
   - 에이전트 완성

부품 사용

08-1 랭체인으로 챗봇 만들기
08-2 LCEL로 체인 만들기 (구성) 기호 -> 표현
08-3 랭체인 도구로 에이전트 만들기
08-4 스트림 방식으로 출력하기
08-5 스트림릿에 구현하기

비교 내용	GPT API 순수사용 이전 방식	Langchain
모델준비	client=OpenAI(api-key) res->model	llm=chatOpenAI(model="")
요청보내기	client.chat.completions.create() messages=[] [{"role":"user","content":"질문"}]	llm.invoke(messages) HumanMessage("질문")
역할 이름	system, user, assistant, tool	SystemMessage, HumanMessage AIMessage, ToolMessage
답변 꺼내기	response.choice[0].message.content	response.content
답변 기록 assistant->append	{"role":"assistant","content":"답" -> messages append	답변기록: AIMessage -> append
대화기억	message에 append	사용: RunnableWithMessageHisgtory

함수설명 (도구)	json 직접 코딩	@tool + docstring(함수 내 설명서) (json 자동생성)
요청서 추출 (LLM->client)	json.load(tool_call.function .argument)	response.tool_calls[0]['args'] dict
도구 결과 돌려주기 (함수실행결과)	{"role":"tool", "tool_call_id":id, "content":결과} message.append() -> LLM	tool.invoke(tool_call) -> messages에 추가까지 됨. LLM에 보내기만 하면 됨.
모델 교체	코딩 전체 수정	chatOpenAI() => chatAnthropic()

08-1 랭체인으로 챗봇 만들기

지금까지 오픈AI의 GPT API를 활용해서 언어 모델을 사용하는 방법을 익혔습니다. 앞서 개발했던 간단한 챗봇을 랭체인을 활용해 다시 개발하면서 랭체인의 사용 방법을 알아보고 편의성은 무엇인지 살펴봅시다.

랭체인이란?

라이브러리: 만들어져 있는 기능들

LLM(AI)

랭체인LangChain은 언어 모델에 기반한 AI 애플리케이션을 쉽게 개발할 수 있도록 도와주는 프레임워크입니다. 기존에는 오픈AI와 같은 언어 모델의 API를 사용해 원하는 기능을 구현하려면 모든 코드를 직접 작성해야 했습니다. 랭체인은 이 작업을 간소화할 수 있는 다양한 도구와 모듈을 제공합니다. 랭체인을 사용하면 요약, 검색, 문서 생성, 질의응답 등 여러 기능을 손쉽게 구현할 수 있으며 복잡한 애플리케이션을 개발할 때도 미리 구축된 모듈을 활용해 개발 속도를 높일 수 있습니다.

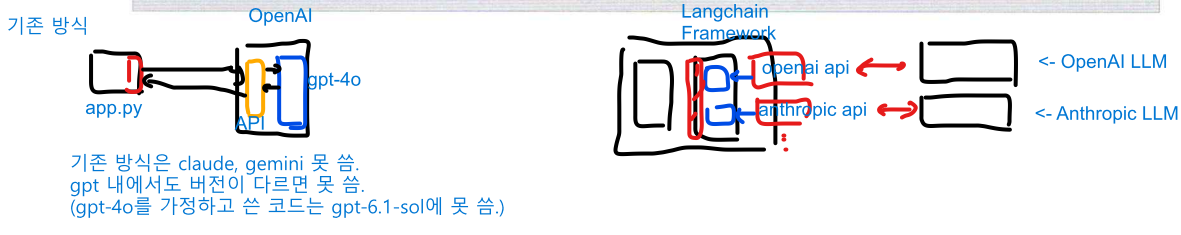
또한 랭체인을 사용하면 다른 언어 모델을 쉽게 교체할 수 있습니다. 구글의 제미나이Gemini, 메타의 라마Llama, 앤트로픽의 클로드Claude와 같은 모델은 오픈AI의 GPT와 API 사용 방식이 달라서 기존 프로그램을 다른 언어 모델로 변경하려면 코드 전반을 수정해야 했습니다. 하지만 랭체인을 사용하면 언어 모델 선언 부분만 수정해서 다른 언어 모델로 쉽게 변경할 수 있습니다. 이 기능 덕분에 특정 모델에 종속되지 않고 다양한 모델의 장점을 활용한 애플리케이션을 개발할 수 있습니다.

Do it! 실습 랭체인과 오픈AI의 GPT API 비교하기

결과 파일: chap08/sec01/langchain_chatbot.ipynb

1. 랭체인을 사용하기 위해 터미널 창에서 랭체인을 설치합니다.

```
(venv) > pip install langchain
```



기존 방식은 claude, gemini 못 씀.
gpt 내에서도 버전이 다르면 못 씀.
(gpt-4o를 가정하고 쓴 코드는 gpt-6.1-sol에 못 씀.)

2. 오픈AI의 GPT API를 사용하기 위해 랭체인에서 오픈AI 모델을 사용할 수 있게 해주는 `langchain-openai`를 설치합니다. 랭체인에서는 GPT뿐만 아니라 제미나이, 라마, 클로드 등 다양한 언어 모델을 사용할 수 있습니다.

```
(venv) > pip install langchain-openai
```

3. langchain_chatbot.ipynb라는 이름으로 파일을 생성하고 `openai` 라이브러리 대신 랭체인에서 제공하는 `ChatOpenAI`를 임포트합니다. 그리고 `gpt-4o-mini` 모델을 사용한다고 선언합니다. 앞에서처럼 .env 파일에 오픈AI의 API 키를 잘 설정해 두었다면 첫 두 줄은 쓰지 않아도 잘 작동합니다.

- langchain_chatbot.ipynb 파일에서 설명할 내용은 랭체인 공식 웹 사이트 튜토리얼의 'Build a Chatbot' 내용을 기반으로 재구성했습니다(https://python.langchain.com/v0.2/docs/tutorials/chatbot/).
- .env 파일 설정은 02-2절 'API 키 관리하기' 실습을 참고하세요.

랭체인으로 첫 챗봇 만들기(1) — langchain_chatbot.ipynb (1)

```
import os
os.environ["OPENAI_API_KEY"] = "YOUR-OPENAI-API-KEY"
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-4o-mini")
```

.env 파일에 API 키가 설정되어 있다면 생략

4. 이어서 랭체인에서 사용자의 메시지를 의미하는 `HumanMessage`를 불러옵니다. 그리고 `model.invoke`에 메시지를 리스트 형태로 전달합니다. 앞에서 오픈AI의 API를 사용할 때는 메시지를 딕셔너리 형태로 작성하여 `{"role": "user", "content", "안녕? 나는 이성용이야."}`와 같이 리스트로 쌓아서 보냈지만 랭체인에서는 다음처럼 작성하면 됩니다.

랭체인으로 첫 챗봇 만들기(2) — langchain_chatbot.ipynb (2)

```
from langchain_core.messages import HumanMessage
model.invoke([HumanMessage(content="안녕? 나는 이성용이야.")])
```

실행하면 다음처럼 결과가 출력됩니다. 앞에서 오픈AI의 API를 사용할 때는 `{"role": "assistant", "content": "안녕하세요, 이성용님! 어떻게 도와드릴까요?"}`라고 출력되었습니다. 반면 랭체인을 사용한 결과는 `AIMessage` 클래스 객체 안에 `content`가 담긴 형태로 반환됩니다.

```
AIMessage(content='안녕하세요, 이성용님! 어떻게 도와드릴까요?', additional_kwargs={'refusal':
None}, response_metadata={'token_usage': {'completion_tokens': 15, 'prompt_tokens':
17, 'total_tokens': 32}, 'model_name': 'gpt-4o-mini-2024-07-18', 'system_fingerprint':
'fp_48196bc67a', 'finish_reason': 'stop', 'logprobs': None}, id='run-07d8a604-65c8-
472e-bc26-02faf8050334-0', usage_metadata={'input_tokens': 17, 'output_tokens': 15,
'total_tokens': 32})
```

랭체인에서는 AIMessage, HumanMessage, SystemMessage와 같은 다양한 메시지 타입을 제공합니다. 여기서 AIMessage는 인공지능이 답변해 주는 역할을 의미합니다. 오픈AI의 API를 활용할 때 role을 assistant로 설정한 것과 같은 개념입니다. HumanMessage, SystemMessage는 각각 user, system과 비슷한 역할을 합니다.

5. 현재 상태에서 랭체인을 통해 GPT와 대화를 이어 나가도 GPT가 이전 대화 내용을 기억하지 못합니다. 다음과 같이 질문해 보면 GPT는 제대로 답변하지 못합니다. `model.invoke`에 전달된 새로운 리스트의 메시지만 활용해 다음 답변을 만들어 줄 뿐이니까요.

랭체인으로 첫 챗봇 만들기(3) langchain_chatbot.ipynb (3)

```
model.invoke([HumanMessage(content="내 이름이 뭐지?")])
```

```
AIMessage(content='죄송하지만, 당신의 이름을 알 수 없습니다. 이름을 알려주시면 제가 기억할 수는
있지만, 현재로서는 알 수 없는 상태입니다. 어떻게 도와드릴까요?', additional_kwargs={'refusal':
None}, response_metadata={'token_usage': {'completion_tokens': 40, 'prompt_tokens':
13, 'total_tokens': 53}, 'model_name': 'gpt-4o-mini-2024-07-18', 'system_fingerprint':
'fp_507c9469a1', 'finish_reason': 'stop', 'logprobs': None}, id='run-0386d621-3fd8-
47a3-b3c4-f2bc46545701-0', usage_metadata={'input_tokens': 13, 'output_tokens': 40,
'total_tokens': 53})
```

Do it! 실습 랭체인으로 멀티턴 대화하기

결과 파일: sec01/langchain_muliturn.py, sec01/langchain_message_history.ipynb

이 책의 03-2절에서는 오픈AI API를 사용해 대화 내용을 담은 리스트에 사용자의 질문과 GPT의 답변을 계속 추가하여 대화를 이어 나갔습니다. 랭체인에서도 같은 방식으로 대화를 이어 나갈 수 있습니다.

대화 내용 리스트에 메시지 추가하기

언어 모델이 맥락에 맞게 답변을 제공하도록 멀티턴 방식으로 랭체인을 구현해 봅시다.

03-2절에서 작성한 코드를 랭체인 스타일로 수정합니다. 기존 코드와 비교할 수 있도록 오픈AI API 스타일의 코드를 남겨 두고 주석으로 표시했습니다. .env 파일에 오픈AI의 API 키를 잘 넣어 두었다면 API 키를 따로 입력할 필요 없이 바로 사용할 수 있습니다.

기존 메시지 리스트에 추가하는 방식으로 구현한 랭체인 멀티턴 — langchain_muliturn.py

```
# from openai import OpenAI
# from dotenv import load_dotenv
# import os
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, AIMessage, SystemMessage

# load_dotenv()
# api_key = os.getenv("OPENAI_API_KEY")  # 환경 변수에서 API 키 가져오기

# client = OpenAI(api_key=api_key)
llm = ChatOpenAI(model="gpt-4o") ①

# def get_ai_response(messages):
#     response = client.chat.completions.create(
#         model="gpt-4o",     # 응답 생성에 사용할 모델 지정
#         temperature=0.9,    # 응답 생성에 사용할 temperature 설정
#         messages=messages,  # 대화 기록을 입력으로 전달
#     )
#     return response.choices[0].message.content  # 생성된 응답 내용 반환

messages = [
    # {"role": "system", "content": "너는 사용자를 도와주는 상담사야."}
    SystemMessage("너는 사용자를 도와주는 상담사야.") ②
]

while True:
    user_input = input("사용자: ")  # 사용자 입력 받기

    if user_input == "exit":       # 사용자가 대화를 종료하려는지 확인
        break

    messages.append(
        # {"role": "user", "content": user_input}
        HumanMessage(user_input)
```

```
    )  # 사용자 메시지를 대화 기록에 추가

    # ai_response = get_ai_response(messages)
    ai_response = llm.invoke(messages)

    messages.append(
        # {"role": "assistant", "content": ai_response}
        ai_response
    )  # 대화 기록에 AI 응답 추가

    print("AI: " + ai_response.content)  # AI 응답 출력
```

1. 랭체인에서 사용할 모델을 정의합니다. 랭체인에서 제공하는 ChatOpenAI를 사용하므로 이전에 사용한 get_ai_response 함수는 이제 필요하지 않습니다.
2. SystemMessage는 앞에서 오픈 AI의 API를 사용할 때 {"role": "system", "content": "적절한 시스템 프롬프트"} 형식으로 작성한 부분을 랭체인 스타일로 처리할 수 있게 해주는 클래스입니다.
3. 또 하나 달라진 점은 llm.invoke로 받은 ai_response를 그대로 메시지 리스트에 추가하는 부분입니다. 앞에서 오픈AI의 API를 사용할 때는 반환된 값에서 content만 추출해서 {"role": "assistant", "content": "GPT의 답변"} 형태로 리스트에 추가했지만, 랭체인에서는 반환값이 이미 AIMessage 형태여서 별도로 content를 추출할 필요 없이 그대로 추가할 수 있습니다.

코드를 실행하면 다음과 같이 대화를 이어갈 수 있습니다. 첫 번째 질문에서 제 이름을 말해주고 두 번째 질문에서 제 정보를 물어보니 기존 대화를 기반으로 잘 답변합니다.

```
사용자: 안녕? 나는 이성용이야.
AI: 안녕하세요, 이성용님! 만나서 반갑습니다. 어떻게 도와드릴까요?
사용자: 나에 대해 알고 있는 정보가 있어?
AI: 죄송하지만, 저는 개인 정보를 저장하거나 접근할 수 없습니다. 이성용님에 대한 정보는 알지 못합니다. 대신, 궁금한 점이나 이야기하고 싶은 주제가 있다면 도와드릴 수 있습니다!
```

랭체인의 메시지 히스토리

앞에서는 멀티턴 대화를 위해 매번 GPT와 사용자의 대화 내용을 리스트나 딕셔너리에 추가하는 코드를 작성해야 했습니다. 랭체인의 메시지 히스토리Message History 기능을 사용하면 멀티턴 대화를 더 쉽게 구현할 수 있습니다.

1. 메시지 히스토리를 단계적으로 이해하기 위해 langchain_message_history.ipynb 파일을 생성하고 코드를 작성합니다.

✦ 이 예제는 랭체인 공식 문서에서 제공하는 메시지 히스토리 사용 방법을 재구성한 것입니다(https://python.langchain.com/v0.2/docs/tutorials/chatbot/).

메시지 히스토리 연습하기(1) langchain_message_history.ipynb (1)

```
from langchain_core.chat_history import InMemoryChatMessageHistory,  ①
from langchain_core.runnables.history import RunnableWithMessageHistory  ②
from langchain_openai import ChatOpenAI  # 오픈AI 모델을 사용하는 랭체인 챗봇 클래스
from langchain_core.messages import HumanMessage

model = ChatOpenAI(model="gpt-4o-mini")

store = {}  ③

# 세션 ID에 따라 대화 기록을 가져오는 함수
def get_session_history(session_id: str) -> BaseChatMessageHistory:
    # 만약 해당 세션 ID가 store에 없으면, 새로 생성해 추가
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()  # 메모리에 대화 기록을 저장하는 객체 생성
    return store[session_id]  # 해당 세션의 대화 기록을 반환  ④

# 모델 실행 시 대화 기록을 함께 전달하는 래퍼 객체 생성
with_message_history = RunnableWithMessageHistory(model, get_session_history)  ⑤
```

① InMemoryChatMessageHistory는 메모리 내에서 메시지를 리스트 형태로 보관합니다. 애플리케이션을 종료하면 대화 내용이 사라지므로 계속 사용하고 싶다면 파일이나 DB에 저장해야 합니다.

② RunnableWithMessageHistory는 모델을 생성할 때 대화 기록을 함께 전달할 수 있도록 하는 클래스입니다.

③ store는 session_id를 기준으로 사용자의 대화 기록을 저장하는 딕셔너리입니다.

④ get_session_history 함수는 RunnableWithMessageHistory 클래스에서 대화 기록을 관리하기 위해 호출되며 session_id에 해당하는 대화 기록을 반환하는 역할을 합니다. store에서 특정 사용자의 대화 기록을 찾고, 만약 해당 session_id가 없으면 새로운 대화 기록 객체를 만들어서 저장합니다. 그리고 해당하는 대화 기록 객체를 반환합니다.

⑤ RunnableWithMessageHistory를 이용하면 대화 내용을 저장하면서 대화를 이어 나갈 수 있습니다. with_message_history를 사용해 RunnableWithMessageHistory 인스턴스를 만듭니다.

2. 이제 ChatOpenAI를 바로 사용하는 model.invoke() 대신 with_message_history()를 실행합니다. 그리고 session_id를 랭체인에서 정한 양식에 맞게 설정합니다. 현재 config 객체에 session_id를 abc2로 설정했습니다. 첫 대화는 이름을 소개하며 시작했습니다.

메시지 히스토리 연습하기(2) langchain_message_history.ipynb (2)

```
config = {"configurable": {"session_id": "abc2"}}  # 세션 ID를 설정하는 config 객체 생성

response = with_message_history.invoke(
    [HumanMessage(content="안녕? 난 이성용이야.")],
    config=config,
)

print(response.content)
```

셀을 실행하면 GPT는 인사를 하며 무엇을 도와줄지 물어봅니다.

```
'당신의 이름은 이성용입니다. 다른 질문이나 도움이 필요하신 것이 있나요?'
안녕하세요, 이성용님! 반갑습니다. 어떻게 도와드릴까요?
```

3. 이어서 이름을 물어보겠습니다. 대화 내용을 잘 기록했다면 제 이름을 잊지 않았겠죠.

메시지 히스토리 연습하기(3) langchain_message_history.ipynb (3)

```
response = with_message_history.invoke(
    [HumanMessage(content="내 이름이 뭐지?")],
    config=config,
)

print(response.content)
```

역시 잘 기억하고 있습니다. 다시 한번 강조하지만 RunnableWithMEssageHistory로 만든 객체를 통해 대화할 때 config로 제공한 session_id에 따라 고유의 대화 내용이 기록됩니다.

```
'당신의 이름은 이성용입니다. 다른 질문이나 도움이 필요하신 것이 있나요?'
```

4. 예를 들어 다음처럼 session_id를 계속 사용하던 abc2가 아니라 abc3로 만들어 입력하면 abc2에 기록했던 대화 내용은 전혀 없는 상태가 됩니다. 따라서 새로운 config로 입력하고 실행하면 GPT는 이전 대화를 기억하지 못합니다.

메시지 히스토리 연습하기(4) langchain_message_history.ipynb (4)

```
config = {"configurable": {"session_id": "abc3"}}

response = with_message_history.invoke(
    [HumanMessage(content="내 이름이 뭐지?")],
    config=config,
)

response.content
```

```
'죄송하지만, 당신의 이름을 알 수 있는 정보가 없습니다. 당신의 이름을 말씀해 주시면 그에 맞춰 대화할 수 있습니다!'
```

5. 다시 session_id를 abc2로 수정하면 기존 대화를 기억하고 잘 대답합니다.

메시지 히스토리 연습하기(5) langchain_message_history.ipynb (5)

```
config = {"configurable": {"session_id": "abc2"}}

response = with_message_history.invoke(
    [HumanMessage(content="아까 우리가 무슨 얘기 했지?")],
    config=config,
)

response.content
```

```
'우리는 당신의 이름이 이성용이라는 것에 대해 이야기했습니다. 추가로 궁금한 점이나 이야기하고 싶은 주제가 있으신가요?'
```

스트림 방식으로 출력하기

이번에는 랭체인으로 만든 챗봇의 답변을 스트림 방식으로 출력해 보겠습니다. 스트림 방식으로 답변을 출력하는 것은 간단합니다. .invoke라고 되어 있던 부분을 .stream으로만 바꿔 주면 됩니다. langchain_message_history.ipynb 파일에 이어서 작성해 보겠습니다.

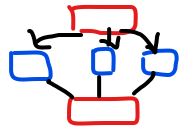
반복문을 사용하여 답변이 생성되는 대로 데이터를 하나씩 출력합니다. 출력 속도가 너무 빠르면 스트림 출력이 되는지 확인하기 어려울 수 있으므로 end="" 대신 end="¦"로 설정합니다.

스트림 방식으로 출력하기 langchain_message_history.ipynb (6)

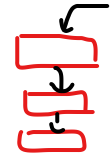
```
config = {"configurable": {"session_id": "abc2"}}
for r in with_message_history.stream(
    [HumanMessage(content = "내가 어느 나라 사람인지 맞춰보고, 그 나라의 국가를 불러줘.")],
    config=config,
):
    print(r.content, end="¦")
```

순차적으로 출력되는 모습을 책에 담을 수는 없지만 응답은 각각 다음처럼 ¦로 구분된 단위로 출력됩니다. 한국의 국가가 '아리랑'이라는 등 잘못된 답변도 있지만 이런 환각 현상은 뒤에서 차차 해결해 나가겠습니다.

```
¦이¦성¦용¦이라는¦ 이름¦은¦ 한국¦에서¦ 많이¦ 사용¦되는¦ 이름¦입니다¦.¦ 따라서¦ 한국¦ 사람¦일¦ 가능
¦성이¦ 높¦다고¦ 생각¦합니다¦.¦ 한국¦의¦ 국¦가는¦ "¦아¦리¦랑¦"¦입니다¦.¦ 맞¦나요¦?¦¦
```

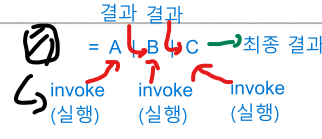


병렬 처리.



연결해주는 명령

08-2 LCEL로 체인 만들기



각 오브젝트(A,B,C)에 invoke() 있어야 함.
Runnable 상속받아 invoke() 오버라이딩.

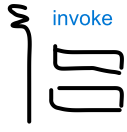
명령
표현 언어

LCEL[LangChain Expression Language]은 랭체인에서 복잡한 작업 흐름을 간단하게 만들고 관리할 수 있도록 돕는 도구입니다. 랭체인에서는 이런 작업 흐름을 연결하는 것을 체인[Chain]이라고 합니다. LCEL을 이용하면 여러 줄로 표현해야 하는 작업 단계를 읽기 쉽게 축약할 수 있으며 스트림 출력 등 여러 작업을 병렬로 처리할 수 있습니다. 이 외에도 더 많은 기능이 있습니다. 직접 코드를 작성하면서 LCEL에 대해 배워봅시다.

Runnable class => invoke

Runnable 상속 받으면
실행 가능한 형태로 바뀜.

상속받지 않으면
파이프라인에 들어갈 수 x
실행될 수 없기 때문.



메소드 구조
시그니처
바디

추상 메소드: 시그니처만 있는 메소드

Do it! 실습 출력 파서와 체인

결과 파일: sec02/lcel.ipynb

먼저 랭체인 기본 문법만으로 챗봇 코드를 작성해 보겠습니다. 03-1절에서 프롬프트 엔지니어링을 배우며 GPT에게 역할을 부여했던 것 기억하나요? 이번에는 랭체인 코드로 역할을 부여한 챗봇을 만들어 보겠습니다. 이어지는 실습에서 LCEL을 단계별로 학습하기 위해 주피터 노트북을 활용하겠습니다.

1. 새 파일 LCEL.ipynb를 만듭니다. GPT-4o-mini 모델을 사용하여 시스템 프롬프트에 '미녀와 야수' 이야기의 미녀(벨) 역할을 맡아 그 캐릭터에 맞게 대답해 달라고 요청해 보겠습니다. 그리고 HumanMessage로 자신을 '미녀와 야수' 이야기에서 빌런 역할인 개스톤이라고 소개하며 저녁 식사를 제안했습니다.

GPT에게 '미녀와 야수' 이야기의 미녀 역할을 부여하고 대화하기 — lcel.ipynb (1)

```
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-4o-mini")

from langchain_core.messages import HumanMessage, SystemMessage

messages = [
    SystemMessage(content="너는 미녀와 야수에 나오는 미녀야. 그 캐릭터에 맞게 사용자와 대화하라."),
    HumanMessage(content="안녕? 저는 개스톤입니다. 오늘 시간 괜찮으시면 저녁 같이 먹을까요?"),
]

model.invoke(messages)
```

LCEL
A | B | C

이 코드를 실행해 보면 챗봇은 그 역할에 맞게 개스톤의 데이트 신청을 거절합니다.

```
AIMessage(content='안녕하세요, 개스톤. 당신의 제안은 고맙지만, 저는 오늘 도서관에서 읽고 싶은 책이 많아서 시간이 없을 것 같아요. 책을 읽는 것이 저에게는 아주 중요한 일이거든요. 이해해 주실 수 있겠죠?', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 61, 'prompt_tokens': 62, 'total_tokens': 123}, 'model_name': 'gpt-4o-2024-05-13', 'system_fingerprint': 'fp_df84d6bd70', 'finish_reason': 'stop', 'logprobs': None}, id='run-25f3b0a0-e232-44a6-887d-23ffbeb91d35-0', usage_metadata={'input_tokens': 62, 'output_tokens': 61, 'total_tokens': 123})
```

파서 : 가공해서 출력

출력된 결과를 보면 AIMessage라는 객체 안에 여러 정보가 포함되어 있습니다. 만약 텍스트 결과만 필요하다면 StrOutputParser를 사용하면 됩니다. StrOutputParse는 랭체인에서 제공하는 다양한 출력 파서output parser 중 하나입니다. 출력 파서란 언어 모델이 반환하는 결과에서 원하는 형식의 데이터를 추출하는 도구입니다. StrOutputParser는 텍스트만 추출하여 반환하며 다른 파서들은 JSON, 숫자 등 특정 형식을 처리할 수 있습니다.

2. 다음처럼 model.invoke로 messages를 사용하고 그 결과를 parser.invoke에 넘겨주면 원하는 텍스트만 출력할 수 있습니다. 여기서는 문자열만 반환하기 위해 StrOutputParser를 사용합니다. 이처럼 출력 파서를 이용하면 반환 형식을 정해 줄 수 있습니다.

StrOutputParser로 텍스트만 반환하도록 수정하기 lcel.ipynb(2)

```
from langchain_core.output_parsers import StrOutputParser

parser = StrOutputParser()

result = model.invoke(messages)
parser.invoke(result)
```

class < 변수 메소드

파서가 가공해서 빼낸다.

print(result.content) 로 해도 답변 출력됨.
parser로 만들어 놓으면 chain으로 연결할 수 있음.

이 셀을 실행하면 다음처럼 문자열만 출력됩니다. 벨은 다시 한번 개스톤의 제안을 정중히 거절합니다.

```
'안녕하세요, 개스톤. 당신의 제안에 감사드려요. 하지만 저는 오늘 아버지를 돌봐야 하고, 또 책을 읽을 시간이 필요해요. 당신도 알다시피, 저는 독서를 아주 좋아하거든요. 다음에 더 좋은 기회가 있으면 좋겠네요.'
```

3. 앞에서 작성한 코드는 GPT 모델에서 결과를 얻고 그 결과를 StrOutputParser를 사용해 텍스트만 추출하는 2단계가 순차로 이어집니다. 이 2단계를 랭체인에서는 LCEL의 체인을 이용해 | 연산자로 한 줄로 간단하게 연결할 수 있습니다.

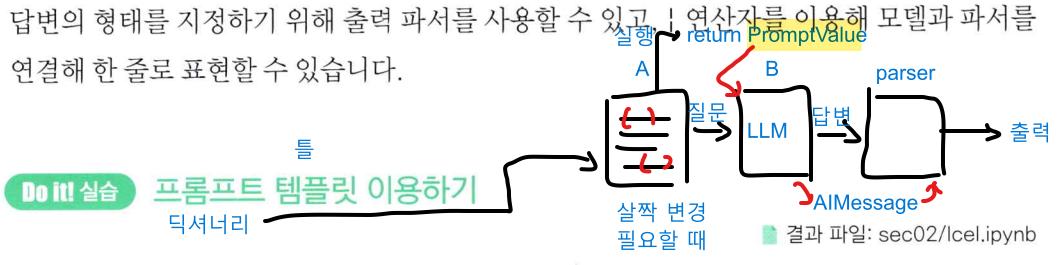
Chain 연산자 |를 이용해 간단하게 수정하기 — lcel.ipynb (3)

```
chain = model | parser
chain.invoke(messages)
```

이 셀을 실행해 보면 벨은 여전히 개스톤의 제안을 거절하고 있습니다.

```
'안녕하세요, 개스톤. 당신의 제안은 고맙지만, 오늘은 책을 읽고 싶어서요. 저는 조용한 시간을 보내는 걸 좋아하거든요. 하지만 당신의 관심에 감사드려요. 혹시 다른 날에 친구들과 함께 모이는 건 어떨까요?'
```

답변의 형태를 지정하기 위해 출력 파서를 사용할 수 있고, | 연산자를 이용해 모델과 파서를 연결해 한 줄로 표현할 수 있습니다.



만약 다른 캐릭터가 벨에게 제안한다면 어떻게 될까요? 매번 messages의 내용을 바꿔서 테스트할 수도 있지만 랭체인의 프롬프트 템플릿prompt template을 이용하면 필요한 부분만 수정하여 실행할 수 있습니다. 이전 파일에 이어서 실습하겠습니다.

1. ChatPromptTemplate을 사용해 대화 중 변수로 만들고 싶은 부분을 대괄호({}) 로 감싸고 변수명을 지정합니다. 여기서는 system_template과 human_template을 문자열로 작성하고 story, character_a, character_b, activity를 변수로 사용합니다. 이를 ChatPromptTemplate 객체인 prompt_template로 생성합니다. 그리고 prompt_template에 .invoke를 사용해 딕셔너리 형태로 항목을 채워 넣습니다. 이때 character_b를 개스톤 대신 야수로 수정했습니다.

Runnable class
- invoke: 입/출력
- stream: 실시간
- ainvoke: 비동기

__or__() 예. A.__or__(B) = A | B
__ror__()

프롬프트 템플릿 이용하기 lcel.ipynb (4)

```
from langchain_core.prompts import ChatPromptTemplate

system_template = "너는 {story}에 나오는 {character_a} 역할이다. 그 캐릭터에 맞게 사용자와 대화하라."
human_template = "안녕? 저는 {character_b}입니다. 오늘 시간 괜찮으시면 {activity} 같이 할까요?"

prompt_template = ChatPromptTemplate([    리스트
    ("system", system_template),
    ("user", human_template),          튜플
])

result = prompt_template.invoke({
    "story": "미녀와 야수",
    "character_a": "미녀",
    "character_b": "야수",
    "activity": "저녁"
})

print(result)
```

출력해 보면 변수로 설정한 부분들이 모두 원하는 값으로 채워진 프롬프트가 생성됩니다.

```
messages=[SystemMessage(content='너는 미녀와 야수에 나오는 미녀 역할이다. 그 캐릭터에 맞게 사용자와 대화하라.'), HumanMessage(content='안녕? 저는 야수입니다. 오늘 시간 괜찮으시면 저녁 같이 할까요?')]
```

2. 이제 랭체인의 | 연산자를 이용해 체인을 구성합니다. 딕셔너리 형태로 채운 내용은 prompt_template을 거쳐 GPT-4o-mini 모델과 StrOutputParser를 통해 최종 결과로 출력됩니다.

체인으로 구성하기 lcel.ipynb (5)

```
chain = prompt_template | model | parser

chain.invoke({
    "story": "미녀와 야수",
    "character_a": "미녀",
    "character_b": "야수",
    "activity": "저녁"
})
```

개스톤의 제안을 계속 거절했던 벨이 야수의 데이트 신청은 단번에 승낙합니다. `story`, `character_a`, `character_b`, `activity` 등 여러 변수를 바꿔 보며 다양한 상황을 테스트할 수 있습니다.

```
'안녕하세요, 야수님! 물론이죠, 저녁을 함께 할 수 있다니 기쁘네요. 오늘은 어떤 요리를 준비하셨나요? 혹시 제가 도울 일이 있을까요?'
```

이 방법을 활용하면 입력 템플릿과 출력 양식을 미리 설정해 두고 다양한 상황을 반복해서 실행하는 데 도움이 됩니다.



08-3 랭체인 도구로 에이전트 만들기

tool
(function)
function calling

1. @tool
2. bind_tools(): LLM에 tool을 보냄.
3. ToolMessage 생성 — name, args

07-2절에서는 펑션 콜링을 이용해 최신 주식 정보를 가져와 대화하는 챗봇을 개발했습니다. 랭체인에서도 이와 유사한 방법을 도구Tools라는 이름으로 제공합니다. 랭체인의 도구 기능을 활용하면 자신이 만든 함수나 다른 사람이 만든 기능을 챗봇에 쉽게 추가할 수 있습니다.

기존에는 tools=[{"a":"b"}]

@tool — docString
함수 -> 명세서 저장

1. 함수

Do it! 실습 @tool 데코레이터로 랭체인에 함수 연결하기

명세서 만들기.

결과 파일: sec03/langchain_tool.ipynb

오픈AI의 API로 펑션 콜링을 사용해 만들었던 현재 시간을 알려 주는 챗봇을 랭체인으로 개발해 보겠습니다. @tool 데코레이터를 사용하면 함수를 도구로 변환할 수 있습니다. 이 데코레이터는 함수를 랭체인에서 외부 도구로 등록하여 언어 모델이 함수를 호출하고 사용할 수 있게 해줍니다. 즉, @tool 데코레이터는 랭체인이 함수의 기능을 언어 모델이 활용할 수 있게 만드는 역할을 합니다.

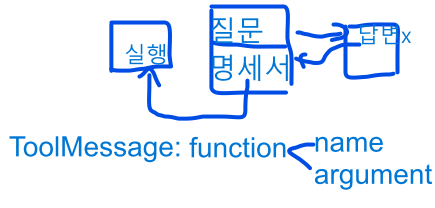
✦ 데코레이터(decorator)는 파이썬에서 함수의 동작을 수정하거나 확장하는 데 사용하는 도구로, 원래의 함수를 변경하지 않고 추가 기능을 덧붙일 수 있게 해줍니다.

1. 랭체인의 도구 기능을 이용하는 방법을 배우는 단계이므로 주피터 노트북 파일을 사용하겠습니다. langchain_tool.ipynb 파일을 만들고 다음과 같이 코드를 입력합니다. 랭체인 스타일로 언어 모델을 선언하고 테스트로 `.invoke()` 메서드를 사용해 간단한 프롬프트를 입력합니다.

언어 모델 선언하기 — langchain_tool.ipynb (1)

```
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage
llm = ChatOpenAI(model="gpt-4o-mini")

llm.invoke([HumanMessage("잘 지냈어?")])
```



이 셀을 실행하면 다음과 같이 답변이 잘 출력됩니다.

```
AIMessage(content='네, 잘 지냈어요! 당신은 어떻게 지내고 계신가요?', additional_kwargs={'refusal
': None}, response_metadata={'token_usage': {'completion_tokens': 18, 'prompt_tokens':
12, 'total_tokens': 30}, 'model_name': 'gpt-4o-mini-2024-07-18', 'system_fingerprint':
'fp_9722793223', 'finish_reason': 'stop', 'logprobs': None}, id='run-e6756b2c-7322-
4bce-80d1-5627c24d9b62-0', usage_metadata={'input_tokens': 12, 'output_tokens': 18,
'total_tokens': 30})
```

앞에서 오픈AI API의 펑션 콜링을 배울 때 만들었던 get_current_time 함수를 가져와서 랭체인의 도구로 활용해 보겠습니다.

2. get_current_time 함수를 랭체인으로 언어 모델에 연결합니다. 함수에 """ """로 표시된 설명은 단순한 주석이 아니라 랭체인에 이 함수의 기능과 입력값, 사용 방법을 알려 주는 문서화 문자열docstring입니다. 앞에서 오픈AI의 API를 이용해 펑션 콜링을 사용할 때는 딕셔너리로 정보를 작성했지만 이제 문서화 문자열로 설명합니다. 또한 기존의 timezone 매개변수 외에도 location 매개변수를 추가하여 어느 지역을 의미하는지를 문자열로 지정합니다. 예를 들어 '부산 몇 시야?'와 같은 질문에 답할 때 timezone은 서울(Asia/Seoul)로 지정됩니다. Asia/Busan 같은 타임존 코드는 존재하지 않으니까요. location은 GPT가 '부산은 현재 00시입니다.'와 같은 답변을 생성하기 편하도록 미리 준비해 두는 용도입니다.

✦ location은 반드시 필요한 매개변수는 아니라고 생각할 수도 있지만 여러 매개변수가 있는 함수를 사용하는 방법을 보여 주기 위해 추가했습니다.

랭체인에 시간을 파악하는 도구 추가하기 — langchain_tool.ipynb (2)

```
from langchain_core.tools import tool
from datetime import datetime
import pytz

@tool  # @tool 데코레이터를 사용하여 함수를 도구로 등록
def get_current_time(timezone: str, location: str) -> str:
    """ 현재 시각을 반환하는 함수

    Args:
        timezone (str): 타임존(예: 'Asia/Seoul'). 실제 존재해야 함
        location (str): 지역명. 타임존은 모든 지명에 대응되지 않으므로 이후 llm 답변 생성에 사용됨
    """
    tz = pytz.timezone(timezone)
```

```
    now = datetime.now(tz).strftime("%Y-%m-%d %H:%M:%S")
    location_and_local_time = f'{timezone} ({location}) 현재 시각 {now} '  # 타임존, 지역명,
현재 시각을 문자열로 반환
    print(location_and_local_time)
    return location_and_local_time
```

3. 이어서 사용할 수 있는 도구를 리스트와 딕셔너리 형태로 만듭니다. 이제 `.bind_tools()` 메서드를 사용하여 기존에 선언한 언어 모델에 도구를 등록할 수 있습니다.

함수 get_current_time을 랭체인으로 llm에 연결하기 — langchain_tool.ipynb (3)

```
# 도구를 tools 리스트에 추가하고, tool_dict에도 추가
tools = [get_current_time,] LLM에 보낼 함수 목록
tool_dict = {"get_current_time": get_current_time,} 실행할 함수 연결

# 도구를 모델에 바인딩: 모델에 도구를 바인딩하면, 도구를 사용하여 답변을 생성할 수 있음
llm_with_tools = llm.bind_tools(tools)
```

4. 이제 준비를 모두 마쳤습니다. `messages`라는 리스트 변수를 만들고 `SystemMessage`와 `HumanMessage`를 담습니다. 메시지가 담긴 `messages` 리스트에 새로 만든 `llm_with_tools.invoke()`를 사용해 답변을 생성합니다. 그리고 GPT의 답변을 `response`에 담아 `messages`에 추가합니다.

사용자의 질문과 도구를 사용해 언어 모델 답변 생성하기 — langchain_tool.ipynb (4)

```
from langchain_core.messages import SystemMessage

# 사용자의 질문과 도구를 사용해 언어 모델 답변 생성
messages = [
    SystemMessage("너는 사용자의 질문에 답변을 하기 위해 tools를 사용할 수 있다."),
    HumanMessage("부산은 지금 몇 시야?"),
]

# llm_with_tools를 사용해 사용자의 질문에 언어 모델 답변 생성
response = llm_with_tools.invoke(messages)
messages.append(response)

# 생성된 언어 모델 답변 출력
print(messages)
```

참고. llm이 함수 이름, argument 가져오므로 invoke로 실행.

출력해 보면 다음과 같이 AIMessage를 반환합니다. 함수를 실행해야 한다고 판단했으므로 content에는 값이 없고 그 대신 tool_calls에는 도구 호출에 필요한 정보가 딕셔너리 형태로 포함되어 있습니다. GPT는 get_current_time 함수를 실행해야 한다고 판단했으며, 이 함수에 필요한 arguments에는 timezone은 'Asia/Seoul'로, location은 '부산'으로 입력해야 한다고 결정했습니다.

```
[
    SystemMessage(content='너는 사용자의 질문에 답변을 하기 위해 tools를 사용할 수 있다.'),
    HumanMessage(content='부산은 지금 몇시야?'),
     AIMessage(content='', additional_kwargs={'tool_calls': [{'id': 'call_Khp6XSqRl8
ns9xJ0Kr4Hwgoj', 'function': {'arguments': '{"timezone":"Asia/Seoul","location":"부산"}',
'name': 'get_current_time'},
(... 생략 ...)
```

이번에는 GPT가 함수 하나만 실행하면 된다고 판단했으므로 호출하는 함수는 response.tool_calls 리스트 안에 1개만 들어 있습니다. 하지만 만약 사용자가 '부산과 뉴욕의 시간 알려줘'라고 요청한다면 함수 2개를 실행해야 한다고 판단할 수 있습니다. 이 경우 tool_calls 리스트에 여러 함수 호출 정보가 들어가며 각각 별도로 실행되도록 for 문을 사용할 수 있습니다.

5. for 문을 사용해 response.tool_calls에서 함수 호출 정보를 하나씩 가져와 반복합니다. 현재는 get_current_time 함수만 도구로 등록해 놓았으므로 selected_tool 변수는 항상 이 함수로만 설정됩니다.

함수 실행 결과 출력하기 — langchain_tool.ipynb (5)

```
for tool_call in response.tool_calls:
    selected_tool = tool_dict[tool_call["name"]]  # tool_dict를 사용해 도구 함수 선택
    print(tool_call["args"])                      # 도구 호출 시 전달된 인자 출력
    tool_msg = selected_tool.invoke(tool_call)    # 도구 함수를 호출해 결과 반환
    messages.append(tool_msg)

messages
```

이 코드를 실행해 보면 마지막에 ToolMessage의 content에 'Asia/Seoul (부산) 현재 시각 2024-09-01 13:06:08'이라는 내용이 출력됩니다.

```
{'timezone': 'Asia/Seoul', 'location': '부산'}
Asia/Seoul (부산) 현재 시각 2024-09-01 13:06:08
[SystemMessage(content='너는 사용자의 질문에 답변을 하기 위해 tools를 사용할 수 있다.'),
 HumanMessage(content='부산은 지금 몇 시야?'),
 AIMessage(content='', additional_kwargs={'tool_calls': [{'id': 'call_7VxJ9DV2mQP6XfAz
gdhLZfBK', 'function': {'arguments': '{"timezone":"Asia/Seoul","location":"부산"}',
(... 생략 ...)
 ToolMessage(content='Asia/Seoul (부산) 현재 시각 2024-09-01 13:06:08 ', name='get_current_
time', tool_call_id='call_7VxJ9DV2mQP6XfAzgdhLZfBK')]
```

6. 현재까지 대화한 내용이 담긴 messages를 llm_with_tools.invoke에 넘기면 현재 시각 정보를 문장으로 답변해 줍니다.

함수 실행 결과를 문장으로 출력하기 — langchain_tool.ipynb (6)

```
llm_with_tools.invoke(messages)
```

```
AIMessage(content='부산은 지금 2024년 9월 1일 오후 1시 6분입니다.', additional_kwargs={'
refusal': None}, response_metadata={'token_usage': {'completion_tokens': 24, 'prompt_
tokens': 192, 'total_tokens': 216}, 'model_name': 'gpt-4o-mini-2024-07-18', 'system_
fingerprint': 'fp_f905cf32a9', 'finish_reason': 'stop', 'logprobs': None}, id='run-
dfedc34e-5855-491a-9e0f-b06037ee42f8-0', usage_metadata={'input_tokens': 192, 'output_
tokens': 24, 'total_tokens': 216})
```

Do it! 실습 파이단틱 이용하기

결과 파일: sec03/langchain_tool.ipynb

파이단틱Pydantic은 입력된 데이터의 유효성과 형식을 검증하고 특정 데이터 형식으로 명확하게 표현할 때 사용하는 라이브러리입니다. 예를 들어 07장에서 만든 주가 정보를 가져오는 get_yf_stock_history 함수는 종목을 의미하는 티커(ticker)와 기간(period)을 매개변수로 받습니다. 이 매개변수의 형식을 파이단틱의 베이스모델(BaseModel)과 필드(Field)를 이용해 더욱 명확하게 정의할 수 있습니다.



이번 실습에서는 파이단틱을 사용해 앞에서 만든 주가 정보 get_yf_stock_history 함수의 입력값 형식을 명확하게 정의하고, 이를 랭체인의 도구로 변환하여 답변을 생성해 보겠습니다. langchain_tool.ipynb 파일에 이어서 작업하면 됩니다.

1. StockHistoryInput 클래스를 만들어서 입력받을 변수의 데이터 형식과 설명을 작성합니다.

pydantic으로 함수 입력값 형식 표현하기(1) — langchain_tool.ipynb (7)

```
from pydantic import BaseModel, Field

class StockHistoryInput(BaseModel):
    ticker: str = Field(..., title="주식 코드", description="주식 코드 (예: AAPL)")
    period: str = Field(..., title="기간", description="주식 데이터 조회 기간 (예: 1d, 1mo,
1y)")
```

2. 그리고 get_yf_stock_history 함수에서 이 클래스 형식으로 매개변수를 받도록 수정합니다. tools와 tool_dict는 새로 만든 get_yf_stock_history 함수를 추가해서 다시 선언했습니다.

pydantic으로 함수 입력값 형식 표현하기(2) — langchain_tool.ipynb (8)

```
import yfinance as yf

@tool
def get_yf_stock_history(stock_history_input: StockHistoryInput) -> str:
    """ 주식 종목의 가격 데이터를 조회하는 함수"""
    stock = yf.Ticker(stock_history_input.ticker)
    history = stock.history(period=stock_history_input.period)
    history_md = history.to_markdown()

    return history_md

tools = [get_current_time, get_yf_stock_history]
tool_dict = {"get_current_time": get_current_time, "get_yf_stock_history": get_yf_
stock_history}

llm_with_tools = llm.bind_tools(tools)
```

3. messages에 HumanMessage로 테슬라의 주가 변화에 관한 질문을 추가하고 llm_with_tools.invoke를 사용하여 답변을 요청합니다. 그 결과를 출력하고 messages에 추가했습니다.

함수 실행 결과 출력하기 langchain_tool.ipynb (9)

```
messages.append(HumanMessage("테슬라는 한 달 전에 비해 주가가 올랐나 내렸나?"))

response = llm_with_tools.invoke(messages)
print(response)
messages.append(response)
```

이 셀의 코드를 실행한 결과는 다음과 같습니다. content=''로 비어 있지만, stock_history_input에 정의한 ticker와 period의 값을 get_yf_stock_history 함수에 넣어 실행해야 한다고 GPT가 판단한 결과가 보입니다.

```
content='' additional_kwargs={'tool_calls': [{'id': 'call_r0epqp08PFgzYDxRrclC43i6',
'function': {'arguments': '{"stock_history_input":{"ticker":"TSLA","period":"1mo"}}',
'name': 'get_yf_stock_history'}, 'type': 'function'}], 'refusal': None} response_metadata=
{'token_usage': {'completion_tokens': 28, 'prompt_tokens': 283, 'total_tokens': 311,
'completion_tokens_details': {'accepted_prediction_tokens': 0, 'audio_tokens': 0,
'reasoning_tokens': 0, 'rejected_prediction_tokens': 0}, 'prompt_tokens_details':
{'audio_tokens': 0, 'cached_tokens': 0}}, 'model_name': 'gpt-4o-mini-2024-07-18',
'system_fingerprint': 'fp_72ed7ab54c', 'finish_reason': 'tool_calls', 'logprobs':
None} id='run-fce155da-6c2b-4b3a-99a0-bfee2a16c919-0' tool_calls=[{'name': 'get_yf_
stock_history', 'args': {'stock_history_input': {'ticker': 'TSLA', 'period': '1mo'}},
'id': 'call_r0epqp08PFgzYDxRrclC43i6', 'type': 'tool_call'}] usage_metadata={'input_
tokens': 283, 'output_tokens': 28, 'total_tokens': 311, 'input_token_details': {'audio'
: 0, 'cache_read': 0}, 'output_token_details': {'audio': 0, 'reasoning': 0}}
```

4. 이제 GPT가 의도한 대로 함수를 실행하고 결과를 메시지로 추가할 수 있도록 다음과 같이 코드를 만듭니다. 위에 출력된 결과를 보면 response 안의 tool_calls에 리스트 형태로 실행할 함수들이 들어 있는 형태였습니다. 따라서 response.tool_calls로 for 문을 이용해 GPT가 계획했던 함수를 모두 실행하고 그 결과들을 messages에 추가합니다. 현재는 tool_calls에서 한 번만 실행하도록 GPT가 결정을 했으므로 한 번만 돌아가게 되어 있습니다.

결과를 메시지로 추가하기 langchain_tool.ipynb (10)

```
for tool_call in response.tool_calls:
    selected_tool = tool_dict[tool_call["name"]]
    print(tool_call["args"])
    tool_msg = selected_tool.invoke(tool_call)
    messages.append(tool_msg)
    print(tool_msg)
```

이 셀을 실행하니 의도한 대로 get_yf_stock_history 함수를 사용한 결과를 잘 가져왔습니다.

```
{'stock_history_input': {'ticker': 'TSLA', 'period': '1mo'}}
content='| Date                       |    Open |    High |     Low |    Close |       Volume |   Dividends |
Stock Splits |\n|:---------------------------|--------:|--------:|--------:|---------:|-------------:|------------:|-
-------------:|\n| 2024-12-11 00:00:00-05:00 | 409.7  | 424.88 | 402.38 |  424.77 | 1.04288e+08 |           0
|            0 |\n| 2024-12-12 00:00:00-05:00 | 424.84 | 429.3  | 415    |  418.1  | 8.77522e+07 |           0
|            0 |\n| 2024-12-13 00:00:00-05:00 | 420    | 436.3  | 415.71 |  436.23 | 8.90002e+07 |           0
|            0 |\n| 2024-12-16 00:00:00-05:00 | 441.09 | 463.19 | 436.15 |  463.02 | 1.14084e+08 |           0
|            0 |\n| 2024-12-17 00:00:00-05:00 | 475.9  | 483.99 | 457.51 |  479.86 | 1.31223e+08 |           0
|            0 |\n| 2024-12-18 00:00:00-05:00 | 466.5  | 488.54 | 427.01 |  440.13 | 1.49341e+08 |           0
|            0 |\n| 2024-12-19 00:00:00-05:00 | 451.88 | 456.36 | 420.02 |  436.17 | 1.18566e+08 |           0
|            0 |\n| 2024-12-20 00:00:00-05:00 | 425.51 | 447.08 | 417.64 |  421.06 | 1.32216e+08 |           0
|            0 |\n| 2024-12-23 00:00:00-05:00 | 431    | 434.51 | 415.41 |  430.6  | 7.26981e+07 |           0
|            0 |\n| 2024-12-24 00:00:00-05:00 | 435.9  | 462.78 | 435.14 |  462.28 | 5.95518e+07 |           0
|            0 |\n| 2024-12-26 00:00:00-05:00 | 465.16 | 465.33 | 451.02 |  454.13 | 7.63664e+07 |           0
|            0 |\n| 2024-12-27 00:00:00-05:00 | 449.52 | 450    | 426.5  |  431.66 | 8.26668e+07 |           0
|            0 |\n| 2024-12-30 00:00:00-05:00 | 419.4  | 427    | 415.75 |  417.41 | 6.4941e+07  |           0
|            0 |\n| 2024-12-31 00:00:00-05:00 | 423.79 | 427.93 | 402.54 |  403.84 | 7.68251e+07 |           0
|            0 |\n| 2025-01-02 00:00:00-05:00 | 390.1  | 392.73 | 373.04 |  379.28 | 1.09711e+08 |           0
|            0 |\n| 2025-01-03 00:00:00-05:00 | 381.48 | 411.88 | 379.45 |  410.44 | 9.54233e+07 |           0
|            0 |\n| 2025-01-06 00:00:00-05:00 | 423.2  | 426.43 | 401.7  |  411.05 | 8.55165e+07 |           0
|            0 |\n| 2025-01-07 00:00:00-05:00 | 405.83 | 414.33 | 390    |  394.36 | 7.56995e+07 |           0
|            0 |\n| 2025-01-08 00:00:00-05:00 | 392.95 | 402.5  | 387.4  |  394.94 | 7.30388e+07 |           0
|            0 |\n| 2025-01-10 00:00:00-05:00 | 391.4  | 399.28 | 377.29 |  394.74 | 6.21275e+07 |           0
|            0 |' name='get_yf_stock_history' tool_call_id='call_r0epqp08PFgzYDxRrclC43i6'
```

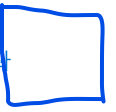
5. 이제 get_yf_stock_history 함수의 실행 결과가 포함된 messages를 llm_with_tools.invoke로 다시 처리하면 자연어 형식으로 최종 답변이 생성됩니다.

자연어로 함수 결과 처리하기 langchain_tool.ipynb (11)

```
llm_with_tools.invoke(messages)
```

```
AIMessage(content='한 달 전인 2024년 12월 11일의 테슬라(TSLA) 주가는 424.77달러로 마감했습니다. 현재 주가는 394.74달러입니다. \n\n따라서, 테슬라의 주가는 한 달 전에 비해 **내렸습니다.**', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 70, 'prompt_tokens': 1527, 'total_tokens': 1597, 'completion_tokens_details': {'accepted_prediction_tokens': 0, 'audio_tokens': 0, 'reasoning_tokens': 0, 'rejected_prediction_tokens': 0}, 'prompt_tokens_details': {'audio_tokens': 0, 'cached_tokens': 0}}, 'model_name': 'gpt-4o-mini-2024-07-18', 'system_fingerprint': 'fp_72ed7ab54c', 'finish_reason': 'stop', 'logprobs': None}, id='run-aa19f728-87bf-4d6a-9717-87e710932b75-0', usage_metadata={'input_tokens': 1527, 'output_tokens': 70, 'total_tokens': 1597, 'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 0, 'reasoning': 0}})
```

8-1번은 질문 -----> 답변(stream)
8-2번은 질문 ----->
<---- tool
실행 -----> 결과
답변 <----



08-4 스트림 방식으로 출력하기

이제 랭체인의 스트림 방식으로 결과를 출력해 봅시다. 08-1절에서 일반적인 대화에서 스트림을 구현하는 방법을 배웠습니다. 하지만 랭체인의 도구Tools를 이용할 때는 GPT의 응답이 단순히 텍스트 형태가 아니라 어떤 함수에 어떤 인자를 넣어 실행할지를 판단해서 딕셔너리 형태로 넘어옵니다. 따라서 스트림 출력을 하려면 랭체인에서도 별도 처리를 해야 합니다. 지금까지 작성한 주피터 노트북 파일을 복습하면서 랭체인에서 스트림 방식으로 출력하는 방법을 익히겠습니다.

Do it! 실습 도구 사용할 때 스트림 출력하기

결과 파일: sec04/langchain_tool_stream.ipynb

앞에서 만들었던 파일을 언어 모델만 있을 때와 도구를 추가했을 때로 나누어 살펴보면서 랭체인 스트림 출력 방식을 익히겠습니다. langchain_tool.ipynb 파일에 이어서 실습하세요.

언어 모델만 있을 때 스트림 방식으로 출력하기

tool 없을 때

도구를 사용하지 않고 언어 모델의 답변을 스트림 방식으로 출력하는 방법은 간단합니다. .invoke() 대신 .stream()을 사용하면 됩니다. 이렇게 하면 답변이 생성되는 즉시 스트림 방식으로 출력됩니다.

코드를 model.invoke()에서 model.stream()으로 수정합니다. 스트림 방식으로 출력되는지 확인하기 위해 길게 답변해야 하는 질문을 추가합니다. 또한 출력할 때 end='¦'를 사용하여 각 단위가 어떻게 출력되는지 확인할 수 있도록 했습니다.

랭체인 도구 사용 시 스트리밍 방식으로 출력하기 — langchain_tool.ipynb (12)

stream하면 잘려들어오니까

```
for c in llm.stream([HumanMessage("잘 지냈어? 한국 사회의 문제점이 무엇인지 이야기해줘.")]):
    print(c.content, end='¦')
```

이런 경우 거의 없을 듯..
다 tool 있을 듯.

이 셀을 실행하면 먼저 만들어진 텍스트가 순차적으로 출력됩니다. 실제로 사용할 때는 end=''로 설정하면 자연스럽게 출력되겠죠.

```
|한국| 사회|의| 문제|점|은| 여러| 측|면|에서| 다양|하게| 나타|납니다|.| 여기| 몇| 가지| 주요| 문
제|점을| 소개|합니다|:

|1|.| **|고|용| 불|안|정|**|:| 청|년|층|을| 중심|으로| 임|시|직|,| 비|정|규|직|의| 비|율|이|
높|아|지고| 있으며|,| 안정|적인| 일|자|리를| 찾|기| 어려|운| 상황|입니다|.| 이는| 경제|적| 불|안|
정|과| 함께| 사회|적| 불|만|을| 초|래|하고| 있습니다|.

|2|.| **|주|거| 문제|**|:| 서울| 및| 대|도|시|의| 집|값| 상승|이| 지속|되고| 있어|,| 젊|은| 세
|대|와| 저|소|득|층|이| 주|거| 공간|을| 확보|하는| 데| 어려|움을| 겪|고| 있습니다|.| 이는| 주|거
| 불|안을| 초|래|하고| 삶|의| 질|을| 저|하시|킵|니다|.
(... 생략 ...)
|6|.| **|노|인| 문제|**|:| 고|령|화| 사회|로| 진|입|하면서| 노|인| 빈|곤|과| 고|독| 문제가| 심|
각|해|지고| 있습니다|.| 노|인| 복|지|와| 관련|된| 정책|이| 더욱| 필요|합니다|.

|7|.| **|교육| 경쟁|**|:| 교육|에| 대한| 과|도|한| 경쟁|과| 스트|레스|는| 학생|들에게| 큰| 부담
|이| 되고| 있으며|,| 이는| 학|업| 성|취|도|와| 정신| 건강|에| 부|정|적인| 영향을| 미|칩니다|.

|이| 외|에도| 다양한| 사회|적|,| 경제|적|,| 문화|적| 문제|들이| 존재|하며|,| 이를| 해결|하기| 위
한| 논|의|와| 노|력이| 지속|적으로| 필요|합니다|.|
```

도구를 추가했을 때 스트림 출력하기

1. 이제 스트림 방식으로 도구 호출 결과를 순차로 받아와 최종 결과를 생성해 보겠습니다. 대화 기록을 리스트에 담는 messages는 새로 선언했습니다.

✦ 다음 코드는 랭체인 공식 문서 'How to stream tool calls'를 참고했습니다(https://python.langchain.com/v0.2/docs/how_to/tool_streaming/).

도구 호출(tool call) 청크 모아서 하나로 만들기 — langchain_tool.ipynb (13)

```
messages = [
    SystemMessage("너는 사용자의 질문에 답변을 하기 위해 tools를 사용할 수 있다."),
    HumanMessage("부산은 지금 몇 시야?"),  ①
]

response = llm_with_tools.stream(messages)  ②
is_first = True
for chunk in response:  ③
    print("chunk type: ", type(chunk))
```

이미 bind_tools() 된 것.

함수선택 args

LLM

invoke 대신 stream

```
    if is_first:
        is_first = False
        gathered = chunk
    else:
        gathered += chunk

    print("content: ", gathered.content, "tool_call_chunk", gathered.tool_calls)

messages.append(gathered)  ④
```

① tools와 연결된 llm_with_tools에서 함수를 호출하도록 '부산은 지금 몇 시인지' 질문합니다.

② 기존 코드에서는 .invoke를 사용했지만 여기서는 .stream을 사용합니다. 단순한 텍스트 답변을 받을 때는 .stream을 사용해도 문제없이 텍스트를 이어 붙여 최종 답변을 만듭니다. 하지만 지금처럼 도구가 연결된 상태에서는 어떤 함수를 어떤 값으로 실행할지에 관한 정보도 스트림 방식으로 반환됩니다. 이 내용을 살펴보기 위해 response를 for 문으로 처리해서 순차적으로 넘어오는 결과를 출력합니다.

③ 이 정보를 처리하려면 하나씩 받은 결과를 모두 이어 붙여서 최종 결과를 만들어야 합니다. 이를 위해 반복문을 시작하기 전에 첫 실행인지 확인하는 is_first 변수를 bool 형태로 만들고, is_first가 False인 경우 gathered를 선언하고 response의 첫 번째 조각을 담습니다. 그리고 그다음부터 gathered에 결과를 계속 추가합니다. 07-3절에서 오픈AI의 API로 펑션 콜링과 스트림 출력을 사용할 때 별도의 함수를 사용한 것과 달리 랭체인에서는 + 연산자를 사용해서 간단히 해결할 수 있습니다.

④ 이렇게 만들어진 gathered는 다음에 GPT에게 요청할 때 사용할 수 있도록 messages에 추가합니다. print(gathered)로 출력할 수도 있지만 너무 많은 정보가 한 번에 출력되는 것을 피하기 위해 간략화했습니다.

실행된 결과를 살펴보겠습니다. 랭체인으로 .stream()을 사용하면 결과가 AIMessageChunk 형식으로 반환됩니다. AIMessageChunk는 개발자가 이전 조각을 쉽게 이어 붙일 수 있도록 도와주는 기능을 합니다. 이 코드에서는 for 문에서 각 조각이 chunk 변수에 담기고 + 연산자를 통해 이 조각들이 gathered에 계속해서 추가됩니다.

```
chunk type:  <class 'langchain_core.messages.ai.AIMessageChunk'>
content:   tool_call_chunk [{'name': 'get_current_time', 'args': {}, 'id': 'call_flsA-
jdi2Dqt92zxDl9iLsrH1', 'type': 'tool_call'}]
(... 생략 ...)
chunk type:  <class 'langchain_core.messages.ai.AIMessageChunk'>
content:    tool_call_chunk [{'name': 'get_current_time', 'args': {'timezone': 'Asia/
Seoul', 'location': '부'}, 'id': 'call_flsAjdi2Dqt92zxDl9iLsrH1', 'type': 'tool_
call'}]
chunk type:  <class 'langchain_core.messages.ai.AIMessageChunk'>
content:    tool_call_chunk [{'name': 'get_current_time', 'args': {'timezone': 'Asia/
Seoul', 'location': '부산'}, 'id': 'call_flsAjdi2Dqt92zxDl9iLsrH1', 'type': 'tool_call'}]
```

맨 마지막 걸로 실행해야 함.

stream()으로 하면 chunk가 붙음.

```
chunk type:  <class 'langchain_core.messages.ai.AIMessageChunk'>
content:    tool_call_chunk [{'name': 'get_current_time', 'args': {'timezone': 'Asia/
Seoul', 'location': '부산'}, 'id': 'call_flsAjdi2Dqt92zxDl9iLsrH1', 'type': 'tool_
call'}]
chunk type:  <class 'langchain_core.messages.ai.AIMessageChunk'>
content:    tool_call_chunk [{'name': 'get_current_time', 'args': {'timezone': 'Asia/
Seoul', 'location': '부산'}, 'id': 'call_flsAjdi2Dqt92zxDl9iLsrH1', 'type': 'tool_
call'}]
```

2. 주피터 노트북의 새로운 셀에서 다음과 같이 + 연산자로 계속 더해진 gathered를 출력해 보면 여전히 AIMessageChunk 타입임을 확인할 수 있습니다. 즉, AIMessageChunk는 계속해서 이어 붙일 수 있는 형태로 유지됩니다.

AIMessageChunk를 계속 더해도 유지되는 타입 — langchain_tool.ipynb (14)

```
gathered
```

```
AIMessageChunk(content='', additional_kwargs={'tool_calls': ... 생략 ...])
```

3. AIMessageChunk는 과거 대화 내용이 담긴 messages에 추가하여 랭체인에서 바로 활용할 수 있습니다. 함수 실행 과정은 스트림 방식으로 출력할 필요가 없으므로 selected_tool.invoke를 사용하여 실행합니다. 이 단계에서는 날씨 API 실행 결과가 tool_msg에 반환되며 그 결과를 다시 message에 추가하고 출력했습니다.

AIMessageChunk에 기존 대화를 추가해서 도구 사용하기 — langchain_tool.ipynb (15)

```
for tool_call in gathered.tool_calls:
    selected_tool = tool_dict[tool_call["name"]]  # tool_dict를 사용하여 도구 이름으로
도구 함수 선택
    print(tool_call["args"])                      # 도구 호출 시 전달된 인자 출력
    tool_msg = selected_tool.invoke(tool_call)    # 도구 함수를 호출하여 결과 반환
    messages.append(tool_msg)

messages
```

실행 결과를 보면 이전에 만들어 두었던 get_current_time 함수의 실행 결과가 마지막에 ToolMessage로 추가된 것을 확인할 수 있습니다.

```
[SystemMessage(content='너는 사용자의 질문에 답변을 하기 위해 tools를 사용할 수 있다.'),
 HumanMessage(content='부산은 지금 몇 시야?'),
 AIMessageChunk(content='', additional_kwargs={'tool_calls': [{'index': 0, 'id':
'call_xtmRs26rZf3ACp26zXHmFPyG', 'function': {'arguments': '{"timezone":"A-
sia/Seoul","location":"부산"}', 'name': 'get_current_time'}, 'type': 'func-
tion'}]}, response_metadata={'finish_reason': 'tool_calls', 'model_name': 'gpt-4o-
mini-2024-07-18', 'system_fingerprint': 'fp_483d39d857'}, id='run-e48280b5-2582-4a1c-
8f62-be97bd176dab', tool_calls=[{'name': 'get_current_time', 'args': {'timezone':
'Asia/Seoul', 'location': '부산'}, 'id': 'call_xtmRs26rZf3ACp26zXHmFPyG', 'type':
'tool_call'}], tool_call_chunks=[{'name': 'get_current_time', 'args': '{"timezone":"Asia/
Seoul","location":"부산"}', 'id': 'call_xtmRs26rZf3ACp26zXHmFPyG', 'index': 0, 'type':
'tool_call_chunk'}]),
 ToolMessage(content='Asia/Seoul (부산) 현재 시각 2024-09-07 17:05:52 ', name='get_cur-
rent_time', tool_call_id='call_xtmRs26rZf3ACp26zXHmFPyG')]
```

4. 이 상태에서 llm_with_tools.stream()을 사용하여 마지막 답변을 생성하고 스트림 방식으로 출력합니다. 이전 실습처럼 end='¦'를 사용하여 출력이 어디서 끊어지고 넘어오는지 확인할 수 있습니다.

도구 사용해 스트림 방식으로 출력하기 langchain_tool.ipynb (16)

```
for c in llm_with_tools.stream(messages):
    print(c.content, end='¦')
```

```
¦부¦산¦은¦ 지금¦ ¦202¦4¦년¦ ¦9¦월¦ ¦7¦일¦ 오후¦ ¦5¦시¦ ¦5¦분¦입니다¦.¦¦
```

이와 같은 방식으로 다른 함수도 추가하여 스트림 방식으로 출력할 수 있습니다. 파이단틱을 사용한 코드도 같은 방식을 활용해 스트림 방식으로 출력하도록 수정할 수 있습니다.

08-5 스트림릿에 구현하기

앞서 주피터 노트북에서 연습한 코드를 스트림릿으로 구현해 봅시다. 새 파이썬 파일을 생성하고 기존 주피터 노트북에서 작성한 코드를 활용해서 단계별로 구현하겠습니다.

Do it! 실습 랭체인 메모리에 기반한 멀티턴 챗봇 만들기

결과 파일: sec05/langchain_simple_chat_streamlit.py

03-3절에서 만든 스트림릿 챗봇 코드인 streamlit_basic.py 파일을 기반으로 랭체인에서도 스트림릿을 활용해 보겠습니다.

1. langchain_simple_chat_streamlit.py 파일을 만들고 `streamlit`과 랭체인에서 필요한 클래스들을 모두 임포트합니다.

필요한 라이브러리와 클래스 임포트하기 — langchain_simple_chat_streamlit.py

```
import streamlit as st

from langchain_openai import ChatOpenAI  # 오픈AI 모델을 사용하는 랭체인 챗봇 클래스
from langchain_core.chat_history import InMemoryChatMessageHistory,  # 메모리에 대화 기록을 저장하는 클래스

from langchain_core.runnables.history import RunnableWithMessageHistory  # 메시지 기록을 활용해 실행할 수 있는 wrapper 클래스
from langchain_core.messages import HumanMessage, AIMessage, SystemMessage
```

2. 기존 코드에서 사이드바를 주석으로 처리하거나 삭제합니다. 03-3절에서 만든 코드는 사이드바를 사용했지만 오픈AI의 API 키를 환경 변수(.env) 파일에서 관리한다면 사이드바가 필요하지 않습니다. 그리고 `st.title()` 안에 원하는 제목을 써줍니다.

사이드바를 주석 처리하고 페이지 타이틀 추가하기 langchain_simple_chat_streamlit.py

```
(... 생략 ...)
# with st.sidebar:
#     openai_api_key = os.getenv('OPENAI_API_KEY')
#     "[Get an OpenAI API key](https://platform.openai.com/account/api-keys)"
#      "[View the source code](https://github.com/streamlit/llm-examples/blob/main/
Chatbot.py)"
#      "[![Open in GitHub Codespaces](https://github.com/codespaces/badge.svg)]
(https://codespaces.new/streamlit/llm-examples?quickstart=1)"

st.title("💬 Chatbot")
```

3. 다음은 streamlit_basic.py에 있던 코드입니다. st.session_state는 스트림릿이 세션 동안 유지하는 상태 저장용 딕셔너리입니다. 만약 messages 키가 아직 st.session_state에 없다면 ["messages"]에 시스템 메시지를 하나 넣어서 초기화합니다.

세션 상태 초기화하기 — messages 키 langchain_simple_chat_streamlit.py

```
(... 생략 ...)
if "messages" not in st.session_state:
    st.session_state["messages"] = [
        SystemMessage("너는 사용자의 질문에 친절이 답하는 AI 챗봇이다.")
    ]
```

4. store 키로 또 다른 딕셔너리를 만듭니다. store는 세션별 대화 히스토리를 담아 두는 공간으로 사용합니다. messages가 전체 메시지 목록이라면, store는 세션 ID별로 나뉘는 대화 이력 객체를 보관하는 저장소 역할을 합니다.

세션 상태 초기화하기 — store 키 langchain_simple_chat_streamlit.py

```
(... 생략 ...)
# 세션별 대화 기록을 저장할 딕셔너리 대신 session_state 사용
if "store" not in st.session_state:
    st.session_state["store"] = {}
```

5. `get_session_history` 함수를 선언합니다. langchain_message_history.ipynb에서 만들었던 함수를 스트림릿에 맞게 약간 수정했습니다. `session_id`를 매개변수로 받아 `st.session_state["store"]` 딕셔너리에서 해당 `session_id`에 대응하는 대화 이력 객체를 가져옵니다. 만약 `session_id`가 아직 없다면 `InMemoryChatMessageHistory()`를 새로 생성하여 저장합니다. 그리고 최종적으로 해당 세션의 대화 이력 객체를 반환합니다. 이렇게 하면 여러 사용자가 동시에 접속할 때 각자 다른 `session_id`를 기준으로 고유한 대화 이력을 추적할 수 있습니다.

get_session_history 함수 선언하기(1) — langchain_simple_chat_streamlit.py

```
(... 생략 ...)
def get_session_history(session_id: str) -> BaseChatMessageHistory:
    if session_id not in st.session_state["store"]:
        st.session_state["store"][session_id] = InMemoryChatMessageHistory()
    return st.session_state["store"][session_id]
```

6. 랭체인의 `ChatOpenAI`를 이용해 언어 모델을 선택하고 `llm` 변수로 선언합니다. langchain_message_history.ipynb 파일과 동일하게 `RunnableWithMessageHistory`를 사용합니다. `config`도 `session_id`를 `abc2`로 지정하여 만듭니다.

get_session_history 함수 선언하기(2) — langchain_simple_chat_streamlit.py

```
(... 생략 ...)
llm = ChatOpenAI(model="gpt-4o-mini")
with_message_history = RunnableWithMessageHistory(llm, get_session_history)

config = {"configurable": {"session_id": "abc2"}}
```

7. st.session_state.messages에는 지금까지 주고받은 대화가 리스트 형태로 저장되어 있습니다. for 문을 통해 각 메시지 내용을 스트림릿의 st.chat_message로 출력합니다. st.chat_message("user")나 st.chat_message("assistant") 등을 사용해 채팅 인터페이스처럼 출력할 수 있습니다.

이전에 저장된 대화 메시지를 UI에 표시하기(1) — langchain_simple_chat_streamlit.py

```
(... 생략 ...)
# 스트림릿 화면에 메시지 출력
for msg in st.session_state.messages:
    if msg:
        if isinstance(msg, SystemMessage):
            st.chat_message("system").write(msg.content)
        elif isinstance(msg, AIMessage):
            st.chat_message("assistant").write(msg.content)
        elif isinstance(msg, HumanMessage):
            st.chat_message("user").write(msg.content)
```

8. streamlit_basic.py 코드에서 표시한 부분을 찾아서 현재 작성 중인 파일인 langchain_simple_chat_streamlit.py에 맞게 수정합니다.

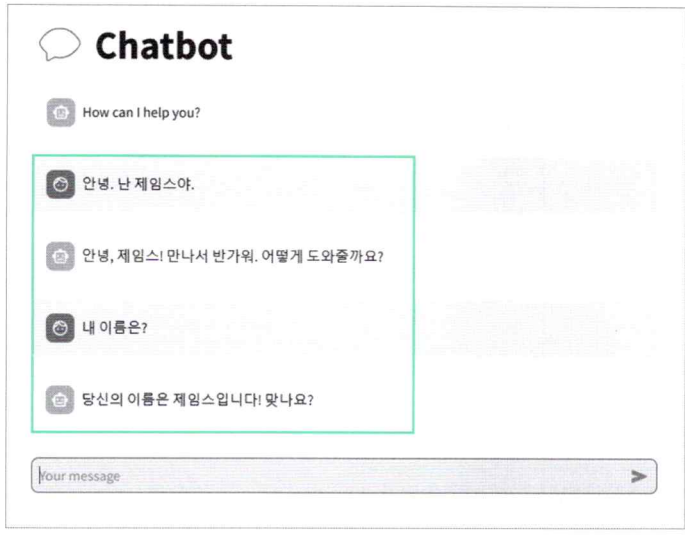
이전에 저장된 대화 메시지를 UI에 표시하기(2) — langchain_simple_chat_streamlit.py

```
(... 생략 ...)
if prompt := st.chat_input():
    print('user:', prompt)
    st.session_state.messages.append(HumanMessage(prompt))
    st.chat_message("user").write(prompt)

    response = with_message_history.invoke([HumanMessage(prompt)], config=config)

    msg = response.content
    st.session_state.messages.append(response)
    st.chat_message("assistant").write(msg)
    print('assistant:', msg)
```

터미널 창에 streamlit run *파일명*.py을 입력해 코드를 실행하면 스트림릿에서 멀티턴 대화를 할 수 있습니다.



9. 마지막으로 스트림릿에서 스트림 방식으로 출력하도록 수정해 보겠습니다. with_message_history.invoke()를 .stream()으로 수정합니다. 추가된 with st.chat_message("assistant").empty()는 07-3절에서 다뤘던 내용입니다. ai_response_bucket를 None으로 선언하고 for 문을 처음 실행할 때 ai_response_bucket에 스트림 방식으로 넘어오는 AIMessageChunk를 담고, 그다음부터는 ai_response_bucket에 계속 이어 붙이는 방식으로 처리합니다. for 문이 종료되면 완성한 ai_response_bucket을 st.session_state.messages에 추가합니다.

스트림 출력하기 langchain_simple_chat_streamlit.py

```
(... 생략 ...)
if prompt := st.chat_input():
    print('user:', prompt)
    st.session_state.messages.append(HumanMessage(prompt))
    st.chat_message("user").write(prompt)

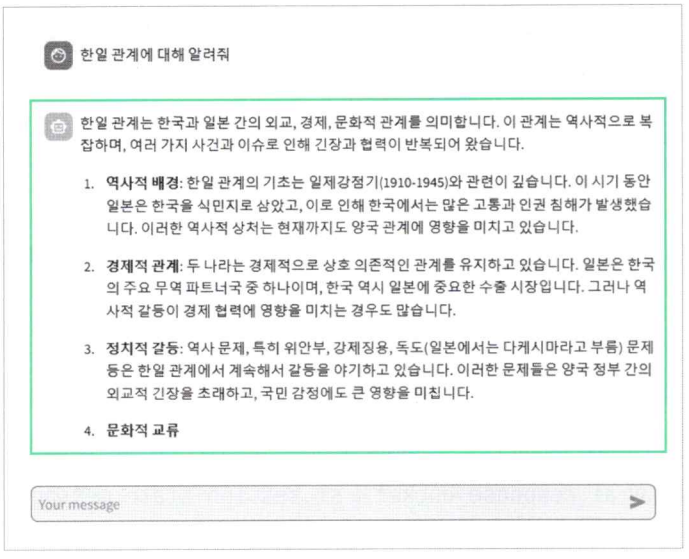
    response = with_message_history.stream([HumanMessage(prompt)], config=config)

    ai_response_bucket = None
    with st.chat_message("assistant").empty():
        for r in response:
            if ai_response_bucket is None:
                ai_response_bucket = r
```

```
            else:
                ai_response_bucket += r
            print(r.content, end='')
            st.markdown(ai_response_bucket.content)

    msg = ai_response_bucket.content
    st.session_state.messages.append(ai_response_bucket)
    print('assistant:', msg)
```

터미널 창에 `streamlit run` *파일명*`.py`을 입력해 코드를 실행하면 결과가 스트림 방식으로 출력됩니다.



Do it! 실습 랭체인 메모리 없이 멀티턴 만들기

결과 파일: sec05/langchain_streamlit_tool_0.py

랭체인의 메모리 기능을 활용하기보다 대화 내용을 리스트로 직접 관리하는 것이 더 간편할 때도 있습니다. 예를 들어 대화 내용을 데이터베이스에 저장하거나 히스토리를 수정해야 할 때는 랭체인의 히스토리를 사용하지 않고 리스트로 직접 관리하는 것이 더 효율적일 수 있습니다. 이번에는 07-3절에서 만들었던 stock_info_streaming.py 파일을 기반으로 스트림릿에서 랭체인의 도구 호출까지 사용하도록 만들어 보겠습니다.

1. 앞서 사용한 랭체인의 히스토리를 사용하지 않고 코드를 작성합니다. 오픈AI의 API에 기반하여 만들었던 코드를 랭체인 기반으로 바꾸고 스트림릿에서 사용할 수 있도록 수정합니다. 여기서 `llm.stream()`으로 사용했으므로 스트림릿 화면에서는 답변이 스트림 방식으로 출력됩니다.

✦ 07-3장에서 만든 stock_info_streaming.py 파일을 수정하여 작성한 코드입니다.

랭체인 메모리 없이 멀티턴 만들기 — langchain_streamlit_tool.py

```
import streamlit as st

from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage, ToolMes-
sage


# 모델 초기화
llm = ChatOpenAI(model="gpt-4o-mini")

# 사용자의 메시지를 처리하는 함수
def get_ai_response(messages):
    response = llm.stream(messages)

    for chunk in response:
        yield chunk

# 스트림릿 앱
st.title("┌ GPT-4o Langchain Chat")

# 스트림릿 session_state에 메시지 저장
if "messages" not in st.session_state:
    st.session_state["messages"] = [
        SystemMessage("너는 사용자를 돕기 위해 최선을 다하는 인공지능 봇이다."),
        AIMessage("How can I help you?")
    ]

# 스트림릿 화면에 메시지 출력
for msg in st.session_state.messages:
    if msg.content:
        if isinstance(msg, SystemMessage):
            st.chat_message("system").write(msg.content)
        elif isinstance(msg, AIMessage):
            st.chat_message("assistant").write(msg.content)
        elif isinstance(msg, HumanMessage):
            st.chat_message("user").write(msg.content)
```

```
# 사용자 입력 처리
if prompt := st.chat_input():
    st.chat_message("user").write(prompt)  # 사용자 메시지 출력
    st.session_state.messages.append(HumanMessage(prompt))  # 사용자 메시지 저장

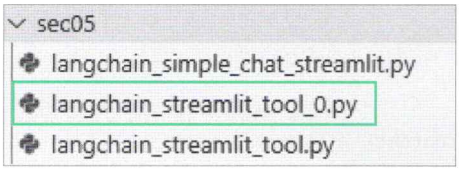
    response = get_ai_response(st.session_state["messages"])

    result = st.chat_message("assistant").write_stream(response)  # AI 메시지 출력
    st.session_state["messages"].append(AIMessage(result))        # AI 메시지 저장
```

터미널 창에 `streamlit run` *파일명*`.py`을 입력해 코드를 실행하면 랭체인 메모리 없이 챗봇이 잘 작동합니다.



2. 지금까지 작성한 코드는 09장에서 다시 활용할 예정이니 따로 저장해 두세요. 앞으로 이 코드에 도구을 추가할 예정입니다. 저는 langchain_streamlit_tool_0.py로 저장했습니다.



Do it! 실습 도구 추가하고 스트림 방식으로 출력하기

결과 파일: sec05/langchain_streamlit_tool.py

현재 시각을 알려 주는 도구를 추가하고 결과를 스트림 방식으로 출력해 보겠습니다.

1. 07-1절에서 정의한 현재 시간을 파악하는 get_current_time 함수를 가져옵니다. 그리고 도구를 연결해서 llm_with_tools로 정의합니다.

Tool 추가하기 langchain_streamlit_tool.py

```
import streamlit as st
from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage, ToolMes-
sage

from langchain_core.tools import tool
from datetime import datetime
import pytz

# 모델 초기화
llm = ChatOpenAI(model="gpt-4o-mini")

# 도구 함수 정의
@tool
def get_current_time(timezone: str, location: str) -> str:
    """현재 시각을 반환하는 함수."""
    try:
        tz = pytz.timezone(timezone)
        now = datetime.now(tz).strftime("%Y-%m-%d %H:%M:%S")
        result = f'{timezone} ({location}) 현재 시각 {now}'
        print(result)
        return result
    except pytz.UnknownTimeZoneError:
        return f"알 수 없는 타임존: {timezone}"

# 도구 바인딩
tools = [get_current_time]
tool_dict = {"get_current_time": get_current_time}

llm_with_tools = llm.bind_tools(tools)
(... 생략 ...)
```

2. 스트림릿에서 도구 호출을 활용한 답변을 스트림 방식으로 출력해 보겠습니다.

랭체인으로 스트림릿에서 스트림 방식으로 출력하기 langchain_streamlit_tool.py

```
(... 생략 ...)
# 사용자의 메시지를 처리하는 함수
def get_ai_response(messages):
    response = llm_with_tools.stream(messages)  ①

    gathered = None  ②
    for chunk in response:
        yield chunk

        if gathered is None:
            gathered = chunk
        else:
            gathered += chunk  ③

    if gathered.tool_calls:
        st.session_state.messages.append(gathered)  ④

        for tool_call in gathered.tool_calls:
            selected_tool = tool_dict[tool_call['name']]
            tool_msg = selected_tool.invoke(tool_call)  ⑤
            print(tool_msg, type(tool_msg))
            st.session_state.messages.append(tool_msg)

        for chunk in get_ai_response(st.session_state.messages):  ⑥
            yield chunk
(... 생략 ...)
```

① get_ai_response 함수에서 llm.stream()을 llm_with_tools.stream()으로 변경합니다.

② 도구를 사용한 결과를 처리할 수 있도록 gathered를 None으로 선언합니다.

③ 스트림되는 응답은 청크 단위로 gathered에 추가되며, 만약 gathered가 None이라면 첫 번째 청크라는 의미이므로 그 값을 gathered에 저장합니다. 이후에는 계속해서 gathered에 값을 추가하는 방식으로 랭체인이 처리됩니다. 이 기능을 이용하면 도구를 사용하는 데 필요한 정보를 계속 합쳐 나갈 수 있습니다.

④ 도구를 사용하지 않는 경우 gathered.tool_calls에는 아무 값도 들어 있지 않습니다. 그러나 도구를 사용해야 한다면 필요한 도구와 매개변수를 확인하고 이를 실행할 수 있습니다.

⑤ 실행된 결과는 tool_msg에 저장되며 기존 대화 내용이 담긴 st.session_state.messages 리스트에 추가됩니다.

⑥ 도구를 다 실행하고 결과를 받은 후, 최종 AIMessage를 받아 오기 위해 다시 한번 get_ai_response() 함수를 사용합니다.

3. 기존의 메시지들을 출력하는 부분에서 ToolMessage도 스트림 방식으로 출력된 결과에 맞춰 스트림릿에 출력되도록 수정합니다.

스트림릿 화면에 메시지 출력하기 langchain_streamlit_tool.py

```
(... 생략 ...)
# 스트림릿 화면에 메시지 출력
for msg in st.session_state.messages:
    if msg.content:
        if isinstance(msg, SystemMessage):
            st.chat_message("system").write(msg.content)
        elif isinstance(msg, AIMessage):
            st.chat_message("assistant").write(msg.content)
        elif isinstance(msg, HumanMessage):
            st.chat_message("user").write(msg.content)
        elif isinstance(msg, ToolMessage):
            st.chat_message("tool").write(msg.content)
(... 생략 ...)
```

코드를 실행하고 대화해 보면 다음처럼 시간을 알려 주는 도구 호출 기능을 활용한 답변이 스트림 방식으로 잘 출력됩니다.

